

FinalProjectReport

Petko Petkov

April 2025

Contents

1	Introduction	2
1.1	Origins of the project	2
1.2	Related Works	2
1.2.1	Workflow 1	3
1.2.2	Workflow 2	4
1.3	Problems with the current system	5
1.4	Alternatives	6
1.4.1	General purpose LLMs	6
1.4.2	Existing tools	6
1.5	Aims and Objectives	8
2	Methodology	8
2.1	Agentic Back-end	9
2.2	Rigid Q&A Flow - Design & Rationale	9
2.3	Incomplete requirements Gathering - Design&Rationale	12
2.4	Judge Pattern For Self Refinement - Design & Rationale	13
2.5	Cumbersome Editing - Design&Rationale	17
2.6	Implementation	19
2.6.1	Slot Filling Schema & ChatBot agent	19
2.6.2	Retrieval Augmented Generation	22
2.6.3	Self-Refining Judge Model	24
2.6.4	Editing and Version History	28
2.7	Rapid Prototyping	30
2.7.1	Design and Rationale	30
2.7.2	Implementation of Streamlit Prototype	31
2.8	Front End and UI	33
2.8.1	Design and Rationale	34
2.9	Implementation (React / Next.js Canvas UI)	37

3	Evaluation	41
3.1	Introduction	41
3.1.1	Purpose of Evaluation	41
3.1.2	Evaluation Goals	42
3.1.3	DSR & ISO compliance	42
3.1.4	Research Questions	42
3.2	Evaluation Design & Methodology	43
3.2.1	Overall Approach	43
3.2.2	Participants	43
3.2.3	Procedure	43
3.2.4	Data Collection Instruments & Metrics	44
3.3	Results	46
3.4	Key Takeaways	46
4	Conclusion	46
5	Appendix	46

1 Introduction

Non-inclusive language in job descriptions has been a deterrent for many candidates. The use of gender-exclusive language significantly decreases the diversity of applicants [1]. With such negatives, it’s surprising that this problem is so widespread; a study by Romano et al[2] found that many processed job descriptions required refinement. JobFair aims to eradicate this problem by using Alice Ashcroft’s extensive research on gender inclusive language, and producing a SaaS product that allows recruiters to generate complete gender inclusive job descriptions.

1.1 Origins of the project

JobFair was co-founded by inclusive language expert Alice Ashcroft and AI integration specialist Josh Bailey (founder of AI consultancy Squash). The venture originated from recurring requests from Alice’s consultancy clients seeking more readily available, on-demand access to her expertise. Identifying an opportunity to meet this need by embedding Alice’s specialised knowledge within an AI system, they collaborated to create JobFair: an LLM-powered SaaS designed to provide expert guidance on inclusive language 24/7.

1.2 Related Works

The current MVP workflow can be broken down into two parts:

1. Generating a job description through a Q&A approach

2. A user uploads an existing job description. Our system then analyses this and produces a list of recommendations to make the existing description more inclusive.

1.2.1 Workflow 1

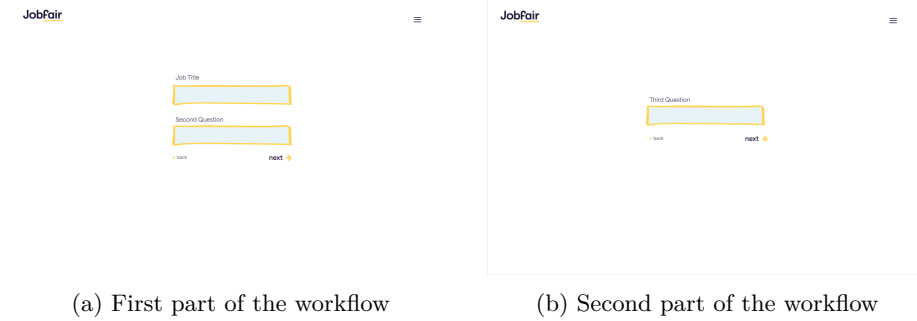


Figure 1: Complete two-step workflow

1 shows the pre-generation part of the workflow. Please note that these are design documents, so 1a has a title field and two question forms instead of one. The user begins by entering the job title for their desired description, followed by answering two preset questions:

1. What skills are essential for your ideal candidate to possess
2. What skills are nice-to-haves

Once the user has answered the two preset questions, an LLM uses the aggregated question-answer pairs to generate a third question for the user to answer about their job description. Finally, a job description is generated using the aggregate of all three question-answer pairs.

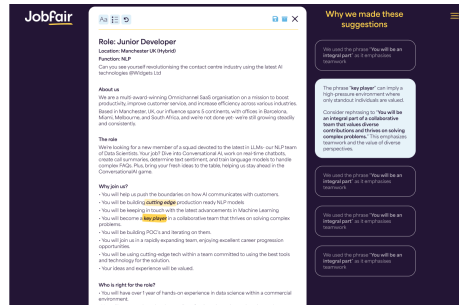


Figure 2: Post generation screen

Figure 2 shows the user interface after job description generation. A transparent side panel that exposes each suggestion's rationale helps recruiters build

an accurate mental model of how the system works, which drives higher trust, lower cognitive load, greater satisfaction, and stronger compliance. By surfacing concise, contextual explanations immediately adjacent to generated copy, JobFair demystifies its inclusive-language engine and transforms every interaction into a learning moment, enabling recruiters to internalise best practices over time. This level of explanation and transparency separates JobFair from competitors who often adopt a black box approach. The explanations provided by JobFair aim to build trust with the user [3]. In addition to this, by providing both “why” and “how” explanations side-by-side, we can significantly improve user satisfaction and perceived usefulness [4]

1.2.2 Workflow 2

The second workflow allows the user to upload an existing Job Description, which the LLM system then analyses.

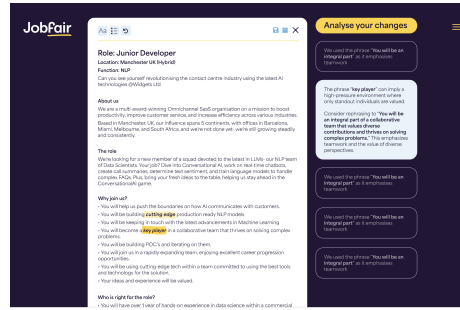


Figure 3: Workflow 2 User Interface

Figure 3 shows the post-analysis interface. As can be seen, there is a list of suggestions on the right-hand side of non-inclusive language. Each suggestion is accompanied by an explanation, similar to workflow one. Again, JobFair emphasises transparency and aims to help users learn about language inclusivity as they use the SaaS. The user can choose whether to apply a suggestion, providing even more control over the final job description. My placement during the second term involved building a scalable backend and architecting a database to support these two workflows. The main issue tackled was developing the agentic backend for workflow 1, which consisted of a straightforward LLM system shown by Figure 4:

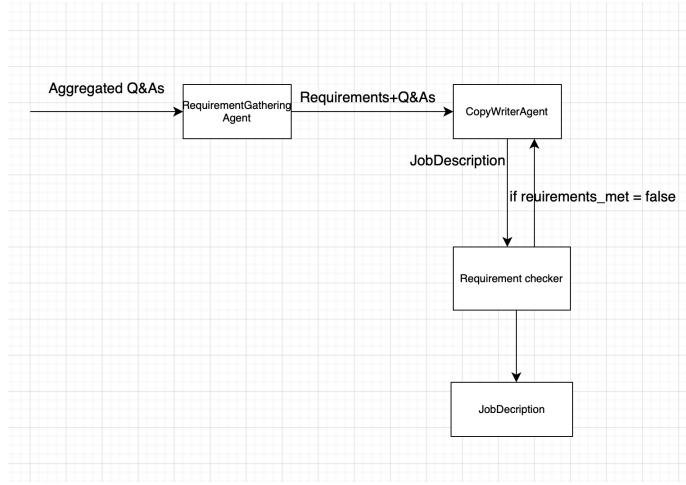


Figure 4: Agentic System for generating a job description

The system can be broken down into three parts

1. A requirements gathering agent extracts a list of requirements from the answered questions
2. The list of requirements in conjunction with the Q&A pairs is provided as context to the copywriter agent, whose goal is to generate a description
3. A requirements checked agent then checks if the description covers all requirements. The description is passed through the system if the agent determines all requirements are met. Otherwise, the copywriter agent continues to rewrite the description until the requirements have been met

1.3 Problems with the current system

Despite achieving the goal of allowing users to create an inclusive job description, the current MVP exhibits several problems that hinder the quality at which it achieves this goal:

1. **Overly rigid interaction model.** All users are funnelled through the same two preset questions, which limits the system’s ability to capture role-specific nuance. This narrow context often results in formulaic, “one-size-fits-all” job descriptions that fail to reflect the actual needs of the position.
2. **Incomplete requirement gathering.** Key sections of a well-crafted posting—such as compensation ranges, benefits, and core responsibilities—can be unintentionally skipped because the static Q&A flow doesn’t prompt recruiters to supply these details. As a result, the output quality

becomes overly dependent on the user’s expertise, leading to inconsistent and potentially misleading job adverts.

3. **Absence of domain guidance.** Most recruiters don’t possess the depth of technical knowledge to understand the nuances in requirements for the role they are hiring. The current system cannot aid the user throughout the requirements gathering stage.
4. **Cumbersome revision process.** After a job description is generated, should the user wish to edit it, they must use a separate editing software and re-import it into Jobfair for further analysis. This process repeatedly fractures the user’s workflow -fragmenting their ”working-spheres”, driving up cognitive load and undermining productivity[5]

1.4 Alternatives

JobFair aims to create a custom LLM experience tailored towards helping recruiters craft inclusive job descriptions from scratch. In turn, there are a few possible alternatives:

1.4.1 General purpose LLMs

The most obvious alternative is to use an all-purpose LLM application like ChatGPT or Google Gemini. Both of these applications have a ”canvas” mode, allowing users to collaborate on work by having an editable panel that enables the user to make changes to the AI’s output, thereby providing a defragmented workflow for recruiters, allowing them to draft and refine a job description within the same interface[5]. However, these LLMS are trained to be a jack of all trades and master of none models are prone to gender and racial bias due to their pre-training data [6]. In addition to this, LLMs struggle with prompt sensitivity, whereby minor prompt tweaks massively swing the LLM’s tone, section order and overall quality of output [7], [8]. Lastly, the black box nature of these models means the user doesn’t understand how the LLM has come to its conclusion. This is a problem JobFair tries to tackle by providing a ”why” accompanying the ”how” when the LLM makes any changes.

1.4.2 Existing tools

AI JobDescription generators : Tools such as workable.com provide a slightly more tailored way of using AI to generate a job description. However, their interfaces tend to capture minimal information from the user, risking a boilerplate output.

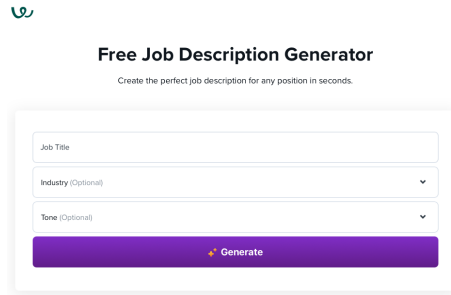


Figure 5: Workable’s user interface

As we can see from Figure 5, workable takes an even more limited question-based approach than job fair. By only asking for a Job Title and Industry, it fails to capture nuanced requirements. In a field like tech, these can vary significantly even if the Job Title is the same. For example Large-scale labour-economics work using Burning Glass vacancies shows “substantial variation in skill requirements, even within narrowly defined occupations” – employers advertising the same role label (e.g., software developer or front-end engineer) ask for markedly different programming languages, frameworks, and tooling stacks, depending on firm strategy and location[9].

In addition to the lack of specificity these competitors provide, many do not focus on gender neutral language whatsoever, and the ones that do, for example Applied(www.beapplied.com), use outdated methods of scoring inclusivity, focusing on masculine and feminine-coded word ratios.

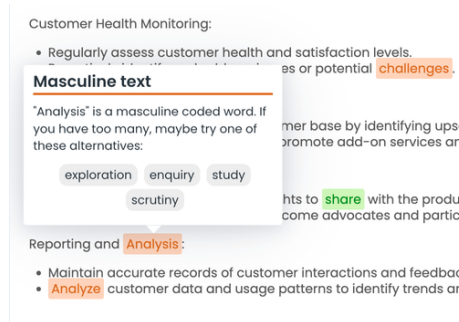


Figure 6: beapplied.com’s tool for inclusive language detection

Recent scholarship shows that simply counting “masculine” vs. “feminine” words—a technique popularised after Gaucher et al. (2011)[10]—is now considered insufficient, weakly predictive in practice, and blind to shifting stereotypes and intersectional bias, so researchers are urging recruiters to move beyond ratio-style gender decoders toward richer, context-aware approaches[11]. JobFair tries to move away from this by using a more nuanced approach, leveraging

the LLM’s power for processing text by providing tailor-made examples by Alice Ashcroft, aiding it to make more nuanced decisions on identifying gendered language.

1.5 Aims and Objectives

As can be seen, there is a clear gap for a transparent, context-aware tool that allows recruiters to create inclusive job descriptions within the same application. JobFair can fill this with a redesign leveraging different practices for optimising LLM output and allowing recruiters to edit/refine a job description seamlessly. The proposed solution aims to deliver a redesigned, full-stack SaaS workflow that eliminates the shortcomings identified in Section 1.3 by coupling an agent-based generation engine with a user-centred interface.

1. **Back-end architecture:** Design, implement and evaluate an agentic pipeline that gathers requirements, retrieves relevant exemplars, iteratively generates inclusive job descriptions, and scores them against predefined criteria.
2. **Front-end experience** – Create an intuitive interface that integrates conversational input and allows post-job description generation and editing, eliminating the need for external editors.
3. **User-driven refinement** – Conduct iterative usability sessions with recruiters and hiring managers; use their feedback to prioritise features and fine-tune both engine and interface.

The rest of this paper will focus on the rationale behind design choices and how the end system aims to improve upon the four identified pitfalls of the previous MVP as stated in 1.3.

2 Methodology

We adopted a Design-Science Research (DSR) approach—building artefacts to solve identified problems—while aligning each build phase with the ISO 9241-210 human-centred design activities. DSR provides the research backbone (problem relevance, rigorous evaluation, transparent contribution) [12], and ISO 9241-210 ensures that even when formal user testing occurs only at the end, every feature is still grounded in recruiter needs [13]. Three internal build cycles addressed the four MVP shortcomings: Cycle 1 produced a slot-filling chatbot plus Retrieval-Augmented Generation (RAG) and an iterative judge to replace the rigid, under-specified flow; Cycle 2 wrapped that engine in a Streamlit proof-of-concept front end for internal assessment; Cycle 3 delivered a React canvas that integrates in-place editing, version history, explainable suggestion cards, and highlight-to-edit—all aimed at removing context-switch costs and boosting trust. Only the fully-integrated artefact underwent a summative study, satisfying the DSR evaluation guideline while meeting ISO’s user-validation requirement.

2.1 Agentic Back-end

The new agentic system fixed all four problems stated in 1.3. The rest of this section will break down the design rationales behind how each was solved, followed by an explanation of the technical implementation.

2.2 Rigid Q&A Flow - Design & Rationale

The first problem was improving the rigid workflow. The only user-provided context in the system is the brief set of answers to the three questions. This isn't enough context for an LLM to generate a good-quality domain-specific response. Research has extensively shown that LLMs' output quality largely depends on rich, structured prompts, without which the LLM is at higher risk of hallucinating and producing a boilerplate response [7]. This is because LLMs will frequently default to outputting boiler-plate tokens in the absence of meaningful context [5]. One option we considered was to extend the existing Q&A, forcing the user to provide the system with more information before a job description is generated. However this causes several issues. As the questionnaire length is extended, the quality of the user's answers also declines, as shown from the research conducted by [14].

In addition to this, the current Q&A approach leaves too much of a burden on the user. The user is left responsible for providing the system with high-quality data, with no guidelines. Since LLMs are highly sensitive to minor variations in prompts slight changes to wording or spelling can produce extreme fluctuations in the quality of the LLMs output [15].

Slot Filling Chatbot Agent: To account for the pitfalls of the static Q&A flow for requirements gathering, we introduced a slot-filling chatbot that collects a rich, structured prompt before generation. A slot is defined as a specific, structured piece of information, such as a list of key requirements or a concise company summary. Slot filling is a well-established strategy in dialogue systems for turning free-text exchanges into complete, structured records [16] and has recently been adopted in requirements-engineering tools such as LadderBot, which more than doubled the amount of concrete user input compared to the alternative information gathering method - static forms [17]. This pre-defined structured output consisting of slots provides the job description generator agent with much richer task-specific context, which is the strongest predictor of factuality and specificity in an LLM's output [18].

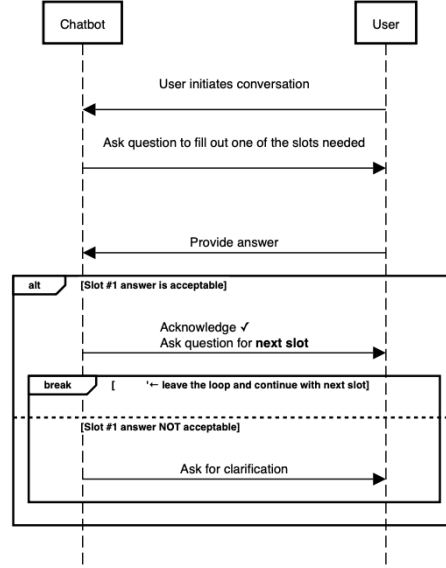


Figure 7: Sequence Diagram of User interacting with Chatbot

Figure 7 shows how the Chatbot would engage with users in conversation until the slots were all filled, guiding them along the way. This guided method of slot filling consistently scores the highest in usability ratings compared to alternative methods [19], i.e the static questionnaire employed by the old MVP. Once all information has been gathered, the slots are passed to the job description generation agent. The rationale behind providing only the slots to the job description generation agent instead of the open ended dialogue between a user and agent was based off the findings of the prompt-SET benchmark, which found that when rephrasing a prompt by as little as 10 tokens swings accuracy by up to 76 percentage points, whereas supplying the same content as key: value *slot tables* cuts that variance by more than 40pp [8],[20]. Therefore with this design we aim to remove the influence of prompt sensitivity on the LLM’s final output.

Retrieval Augmented Generation (RAG): Even with a complete slot table, a large-language model risks reverting to generic phrasing if it has no concrete linguistic examples for the target domain [21]. Several methods for providing this context were considered for this project:

1. **Domain Fine-tuning:** One common approach to specialise an LLM is full domain fine-tuning. This involves retraining the entire model on a large corpus of domain-specific data (in this case, inclusive job descriptions and related literature). While this can lead to deep domain adaptation, we concluded that full fine-tuning would be overkill for this project. The computational resources, time investment, and the sheer volume of high-

quality, meticulously curated data required for a full fine-tune [22] were beyond the practical scope of the agile development needs of JobFair’s MVP. This is an option job fair could consider in the future, if the business is thriving and can invest in creating a well-curated dataset. Since all the weights would be retrained, if the dataset is of poor quality, the model will lose its ability to generalise, leading to a disastrous phenomenon called “catastrophic forgetting”. Whereby the model can lose its generalisation capabilities [23].

2. **Top-Layer Fine-tuning:** A more lightweight alternative is top-layer fine-tuning or other Parameter-Efficient Fine-tuning (PEFT) methods, such as Low-Rank Adaptation (LoRA). These techniques involve updating only a small subset of the model’s parameters, making them significantly less resource-intensive than full fine-tuning. Another promising PEFT method considered was Kahneman-Tversky Optimisation (KTO)[24], which is particularly interesting as it can leverage simpler preference datasets (e.g., “good” vs. “bad” examples) rather than requiring extensive human-written instructions or demonstrations. While these PEFT methods offered a more feasible path, they still needed a high-quality dataset for fine-tuning, even if smaller. Although methods like KTO simplify dataset requirements, curating even a modest dataset of appropriately labelled inclusive and non-inclusive job description examples, or preference pairs, would demand considerable upfront effort to ensure its quality and representativeness. Given the project’s timeline, we explored other methods to produce a more noticeable result while aligning with our constraint of lacking a high-quality labelled dataset.
3. **Retrieval Augmented Generation(RAG):** The third alternative, and the approach ultimately chosen for this project, is Retrieval Augmented Generation (RAG). This technique offers a distinct method for enhancing LLM outputs by dynamically providing external knowledge at inference time. Instead of embedding domain-specific knowledge directly into the model’s parameters through fine-tuning, RAG connects the LLM to an external data source—in our case, a corpus of job descriptions. When a query is made, the RAG system retrieves relevant documents or passages from this corpus. Then, it provides this retrieved information as additional context for the LLM alongside the original prompt. The LLM then uses this augmented prompt to generate its response. RAG was selected due to its strong science-backed data and extensive research demonstrating its effectiveness in grounding LLMs, reducing hallucinations, and improving generated content’s factual accuracy and specificity [25]. Key advantages that made RAG suitable for JobFair included its flexibility (the knowledge base can be updated without retraining the LLM), potential for transparency (it’s often possible to inspect retrieved documents), and a reduced immediate dependency on large, curated fine-tuning datasets for the core model.

Due to the above reasoning, we implemented a lightweight RAG layer that feeds the model ways of phrasing the acquired slot information from a corpus of 400 job descriptions. RAG has been shown to *raise specificity*-Lewis*et al.* reported a 12-point factual F gain on knowledge-intensive QA when retrieved passages were concatenated to the prompt[26]. Follow-up work by Shuster [27] found a 29% drop in hallucinations on the Wizard-of-Wikipedia benchmark [?].

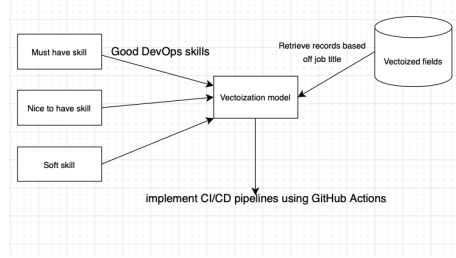


Figure 8: High Level Diagram of RAG usage in the project

Figure 8 shows a very high-level example of the implementation used in this project. As can be seen, because we retrieve *type-matched* requirements for each slot, the model is handed domain-specific verbs and noun phrases “*implement CI/CD pipelines using GitHub Actions*” instead of the generic “good DevOps skills”—thereby eliminating the formulaic phrasing produced by the MVP. This dual-pronged method of using slots together with RAG allows the LLM to learn semantic representations of slots which are transferable across domains and robust to potentially misaligned schemas [28]

2.3 Incomplete requirements Gathering - Design&Rationale

By implementing a ChatBot interface, we also aimed to solve the problem of unstructured requirements gathering present in the old MVP workflow. Unstructured job adverts make it hard to match candidates accurately [29]; NLP analyses of over 42 million unstructured postings report significantly lower retrieval precision when listings lack an explicit schema [30]. The current MVP’s Q&A workflow extracts only three user inputs - job title, must-have skills, and nice-to-have skills- plus a single AI-generated question, which suffers from prompt sensitivity as discussed in the section 2.2. This approach led to two problems:

1. There is a lack of mandatory fields that should be included as part of a “good” job advertisement:
 - (a) A title that indicates the seniority position of the job [31]
 - (b) Specific key responsibilities and tasks [32]

- (c) Compensation and benefits section, which includes pay information, boosting transparency, which in turn reduces gender and minority pay gaps [33]
- (d) Work Location and Flexibility [34]: Many companies have started offering flexible working patterns such as remote or hybrid. The current MVP doesn't encourage recruiters to advertise this.

This project includes all of these as part of the chatbot's "slots".

2. The current MVP burdens the user, leading to inconsistent output quality. For example, the quality of the third AI-generated question is entirely reliant on how much information the user has provided; the system provides no guidance in encouraging the user on how much information they should provide or guidance on how to structure it, other than the question itself, which serves as a vague prompt. This sort of system, which lacks structured, guided interfaces, forces the user to make quality-affecting decisions, increasing the risk of failure [35]. The chatbot interface for requirements gathering removes this burden by having the agent ask for clarification or elaboration from the user whenever more detail is required.

A counterargument to the implementation of this project is simply adding more form fields to the MVP, which the user must answer, essentially mimicking the "slots" implemented in this project. However, the method of collecting user information using "forms" has been proven to score lower in usability and user satisfaction when compared to chatbots [36]. In addition, as mentioned previously, ChatBots for requirements gathering remove the "burden on the user" as the agent will continue to prompt for information it still needs.

Lastly, the current system is much easier to alter. For example, as new research comes out on what should constitute a job description, e.g adding a new slot. The only alterations that need to be made are within the structured output of the LLM agent, while with the MVP, the front-end interface would need to change. This leads us to the conclusion that the new chatbot implementation should be better at gathering requirements and providing the job description generator agent with more context, as well as being easier to extend.

2.4 Judge Pattern For Self Refinement - Design & Rationale

Once the mandatory slots are filled and contextual examples with RAG have been gathered, the job description generator agent produces a first draft, which is evaluated by a Judge Agent.

Sequence Diagram of Job-Description
Writer interacting
with Judge Agent

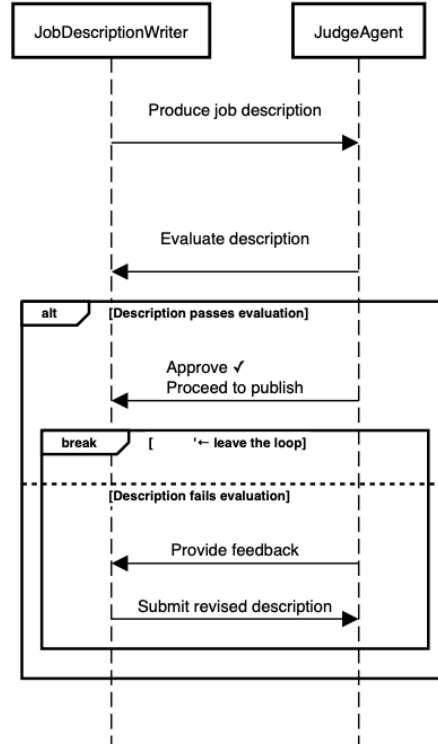


Figure 9: Sequence Diagram of Judge Agent flow

The judge agent’s goal is to grade the output of an LLM against a marking criterion and provide feedback allowing the original agent to iteratively improve its output, as shown by figure 9. The judge then produces a binary decision of **pass** or **fail**. The primary rationale for this inclusion is to remove the negatives of one-shot prompting. The rest of this section will look at the drawbacks of one-shot learning and how that motivated the design of a judge agent for the new system, concluding with a rationale for the marking criteria used by the agent.

One Shot Learning One-shot generation elicits a complete answer from the model in a single prompt pass, with no intermediate critique or opportunity for self-correction. This way of prompting an LLM still produces hallucinations or biased phrasing, even when richer prompts from RAG and the implemented slots in Section 2.2 are used:

1. A paper from Jiang et al.[37] on that knowledge-augmented LLM’s still

often “ignore the given knowledge and hallucinate the answer”, especially if the retrieved context is irrelevant or the model fails to use it. Their work on a verification module shows that without an active “checking” agent, an LLM might produce incorrect outputs despite having the correct facts on hand. This justifies adding a Judge agent to cross-check factuality rather than relying on retrieval alone.

2. Although structured slot prompts and RAG narrow the variance space, prompt-sensitivity cannot be eliminated. Small lexical changes in a user’s wording can still make the slot extractor emit slightly different JSON, which in turn nudges the retriever toward a different exemplar set. Ensemble-prompting and chain-of-thought (CoT) reasoning reduce, yet never abolish this residual fragility by averaging multiple prompts or forcing the model to externalise its reasoning [38]. Our Judge pattern delivers the same robustness in situ: it externalises the model’s latent reasoning into an explicit 5-factor rubric and then iteratively refines the draft until that rubric is satisfied, effectively giving the pipeline an automated Cot-style feedback loop.

Another, more practical, risk of low-fidelity one-shot output is editorial drag on the recruiter. LLM’s struggle with generating content and adhering to a required structure simultaneously [39]. This can lead to a draft that is overly verbose, biased, or missing mandatory sections, forcing the user to:

1. manually clean up the LLM’s output
2. restart generation

Both contradict our goal of a seamless in-context workflow set out in 1.5. Empirical HCI work on fragmented work spheres shows that every forced context-switch (here: from “review” to “rewrite”) elevates mental workload and depresses productivity [40]. Even worse, recruiters will be using our product because they lack deep knowledge of creating an inclusive job description. As a result, a poorly generated draft with subtle gender biased flows could be missed entirely by the recruiters. By having a judge, we introduce a self-correction loop, whereby the agent can recursively improve its output based on structured critique. This method has been shown to significantly increase the quality of the LLM’s output both in terms of factual accuracy/coherence [41], and human preference [42]. With the above justification in mind, the following section will discuss the design and rationale behind the judge agent’s marking criteria.

Judge Agent Rubric The Judge computes a **10-point score** from five categories, with two hard gates (Inclusivity ≥ 1.5 , FRE ≥ 60). Table 1 details the sub-criteria, point splits, and the empirical evidence that motivated each weight.

Table 1: Sub-criteria, weights, and rationale for each rubric bucket.

Bucket (max pts)	Sub-criterion (pts)	Rationale for inclusion
Clarity & Structure (2.0)	Clear headers (0.5)	Improves scanning speed of web users 2013 [43]).
	Logical flow (0.5)	Reduces cognitive load in task sequences (GOV.UK Content Design [44]).
	Bullet lists ≤ 7 (0.5)	Uses Miller’s 7 ± 2 rule to avoid overloading working memory.
	Consistent formatting (0.5)	Predictable layout aids comprehension/accessibility (WCAG 2.2 [45]).
Technical Accuracy (2.0)	Role-skill match (0.7)	Ensures the generation agent doesn’t hallucinate unrealistic requirements; flagged by Indeed Hiring Lab 2022.
	Stack specificity (0.7)	Vague stacks raise hallucination risk; precise tokens cut error (Jiang et al. 2023 [37]).
	Certifications justified (0.6)	Over-spec drives self-selection bias and legal risk (CIPD 2021 survey).
Inclusivity (2.0) <i>gate</i>	Gender Language	This is one of the main issues JobFair tries to tackle, gendered language has been proven to reduce diversity and the number of applicants.
	Readability:FRE ≥ 60 (0.8)	Texts with higher FRE scores were found to be easier to read and quicker to understand [46], with thresholds of above 60 being considered ideal for broad comprehension, therefore broadening pool of candidates
Completeness (2.0)	Mandatory sections present & detailed (2.0)	Ensures the description uses all slot fields.
Engagement (2.0)	Compelling “About Us” (0.7)	Strong mission narrative boosts applicant numbers[47]).
	Growth narrative (0.7)	Clear training/career-path information correlates with higher acceptance rates (LinkedIn Talent Trends 2022).
	Authentic tone (0.6)	Avoiding jargon increases perceived warmth/trust (HR Future 2021 buzzword survey; Lagowska 2023 [48]).

Point allocation. Each bucket totals 2 pts to give equal macro-weighting, while sub-weights mirror the empirical severity of their failure modes (e.g. 1.2 pts for biased language because it severely decreases applicant diversity [1]). The 7/10 pass bar we set was deduced by trial and error as we tried to balance job description quality and speed. From internal testing, we found that at a threshold of 7/10, the job descriptions produced were acceptable. Thresholds higher than this required too many iterations of the write, judge, refine cycle 2.6.3.

Hard-gate justification. Biased or unreadable copy undermines *JobFair*’s DEI mission, hence Inclusivity and Readability are non-negotiable. A FRE 60 score maps to an 8th-grade reading level—recommended by both UK Gov Plain-English guidance and the US CDC health-literacy standards.

Extensibility. Because the rubric is encoded as part of the system prompt for the agent, we could easily change the criteria to fit with new emerging research. The derived rubric is a good start based on empirical evidence.

This concludes the pre-generation part of the agentic system. In the next section, we will discuss design choices for the post-generation phase/editing, after which we will conclude with the implementation of this cycle.

2.5 Cumbersome Editing - Design&Rationale

Limitations in the MVP Editing is not a peripheral convenience but a core phase of every creative workflow: writers revise early drafts to refine structure and voice, designers iterate prototypes to converge on user needs, and engineers refactor code to improve quality. Multiple studies show that meaningful creativity emerges only through repeated cycles of draft-and-revision, each allowing practitioners to externalise ideas, receive feedback, and progressively improve the quality of their work [49]. Modern creative teams rely on dedicated version-control infrastructure precisely because it captures fine-grained change history, enables safe experimentation, and supports accountability during collaborative editing [5]. The MVP breaks this iterative improvement loop by forcing recruiters to export a draft, make edits elsewhere, and then re-upload the file to our system. Consequently, the very functionality recruiters need most post-generation remains outside the system’s boundary.

Design Objectives From section 2.5 we can deduce the following design goals for the editing part of the user workflow:

MVP Limitation (from 2.5)	Design Goal
No in-app revision loop	Enable in-context editing without leaving the SaaS UI
No change provenance	Provide persistent, navigable version history
Risky experimentation	Offer safe “branch-and-revert” workflow to encourage exploration

Table 2: A table of old MVP editing flaws

Our first iteration of the editing flow bundled all three of these into a single Editing Agent, which followed a similar rationale to the ChatBot agent, by allowing the users to make edits through conversing with the LLM

Editing Agent The editing agent provides recruiters with an *in-context, conversational* workspace that removes the round-trip to an external word-processor and closes the revision loop identified in Table 2. Conversational UIs lower extraneous cognitive load by allowing users to express intent in natural language instead of manipulating complex toolbars or modal dialogues [50]. Crucially, the agent *maintains conversation history* across turns, similarly to the design of the Chatbot Agent discussed in section 2.2. Context-aware dialogue models that cache prior messages generate more coherent, on-topic responses and require users to re-enter less information than single-turn systems [51]. By providing the Agent with conversational context of the job descriptions it has produced, we allow it to use previous versions based on the user’s request, e.g, the user says to use the introduction of version two but the style of writing in version 3. The agent will be able to combine these versions, providing the user with more control over making creative changes to job descriptions mirroring the “palette-of-versions” interaction documented in Upwelling’s writer-centric VCS [52] and in Sterman et al.’s study of creative practitioners who reuse earlier iterations to fuel new ideas [53]. Although a linear, save-as-you-go version log might appear sufficient, Chan and Schunn’s large-scale study of conceptual genealogies shows that creativity gains emerge not from *single* “far” recombinations but from *second-order* iteration—i.e. building on ideas that themselves already integrate distant concepts [54]. In their data, first-generation far combinations had, on average, no positive effect on overall concept quality, whereas later generations that *iterated on those far combinations* produced reliably higher-quality, more novel solutions. Our version-history layer operationalises exactly that mechanism: every commit is addressable, and the editing agent can weave fragments from earlier, divergent drafts into a new composite (“use the introduction from v2 and the inclusive tone from v3”). By turning old commits into reusable building blocks rather than static snapshots, the system enables the indirect, multi-step recombination pathway that Chan & Schunn identify as critical for creativity, thereby aligning our design with empirical evidence while still satisfying the safe-exploration goal in Table 2.

2.6 Implementation

The following section will focus on the concrete implementation of the backend system, based on the design choices stated above. The backend was constructed entirely in Python. Although Python is less efficient than compiled languages like C++, or RUST for backend development, particularly in terms of execution speed [55, 56, 57], it is the language with the best support for constructing LLM-based applications [58]. The framework used for agent orchestration was PydanticAI, a lightweight dependency injection-based framework allowing agents access to tools and, most importantly, output validators. Although using the LLM's API would have offered faster response times, this framework is model agnostic, allowing us to swap out models during testing easily. With AI being such a quickly evolving field, the ability to swap out to the newest model with ease was a huge consideration point. The rest of this section will discuss the technical implementation of the new agentic system's components, in order:

1. Chat Bot Agent
2. RAG
3. Self-refining Judge Model
4. Post Generation editing and version history control

2.6.1 Slot Filling Schema & ChatBot agent

The Chatbot Agent is the biggest change from the old MVP product; its functionality can be broken down into three components

1. Structured Output for Slots:

```
from pydantic import BaseModel, Field
from typing import List

class JobRequirements(BaseModel):
    job_title: str
    seniority_level: str = Field(description="Seniority level for the specified role")
    responsibilities: str = Field(
        description="The responsibilities of the job, i.e. day in the life of the role"
    )
    job_requirements: Requirements
    job_salary: str = Field(description="The salary range for the job")
    job_location: str = Field(description="The location of the job")
    job_type: str = Field(
        description="The type of job (e.g. full-time, part-time, contract, etc.)"
    )
```

The Chatbot agent uses Pydantic models to provide structured output; more specifically, its output is of type `JobRequirements`. The Pydantic library allows for complex relationships between classes. It provides the agent with context as to what’s required in each section, e.g. job salary has a description telling the agent to input “The salary range for the job”. Using this method, we ensure consistency in the agent’s final output, a problem that has plagued LLMs since their conception [59].

2. **System Prompt:** Pydantic enables the definition of a system prompt alongside a user prompt. The system prompt, configured by the developer, serves as primary instructions that strongly influence the LLM’s behaviour. Thereby reducing susceptibility to prompt injection attacks [60].

```
CHAT_AGENT_PROMPT = """You are an expert HR chatbot focused on
    gathering the following information from the user:
YOU NEED TO GATHER THE FOLLOWING INFORMATION:
- Role title and level
- Essential technical skills (max 7)
- Essential soft skills (max 5)
- Nice-to-have skills
- Responsibilities
- Job Salary
- Job Location
- Job Type
CRITICAL:
-IF you don't have all the necessary information, return a string
    as your response. This response should be a message to the user
    asking for more information.
-IF you have all the necessary information, return a
    JobRequirements object.
-Ask one question at a time from the user.
-Keep track of what information you have and what you still need
    using the message history context.
-Use inclusive language in your questions.
"""
```

Above is the carefully crafted system prompt. We utilised best prompt engineering practices by providing context to the agent at the start of the prompt [18]. The prompt is well structured with proper spacing, aiding the model with correct tokenisation and processing of the prompt. Lastly, we’ve ensured the prompt is clear and instructional, with two pathways the model can take based on whether the slots have been filled, i.e. if all slots are filled, return a `JobRequirements` object, if not, provide useful feedback to the user. Having a clear, structured system prompt is paramount for the quality of the LLM’s output. Especially for solving complex multi-step reasoning tasks [61].

3. **Memory and Conversation History:** Pydantic agent calls are sin-

gle instance calls to the specified LLM’s API, meaning by default the agent cannot remember context from previous calls. The framework has a straightforward way of implementing this; after running an agent, you can access the messages exchanged during that run from the *result* object [62].

However, the *result* object encompasses more than just user and assistant messages. It also includes system-level components such as *SystemPromptPart* and tool call metadata, which are primarily intended for internal orchestration and guiding the LLM’s behaviour [62]. While these elements are crucial for the agent’s operation, they are not directly relevant to the conversational context we aim to maintain. Incorporating them into the conversation history can introduce noise, potentially leading to less coherent or contextually inappropriate responses from the LLM.

```
def filter_system_prompts(messages: List[ModelMessage]) -> List[
    ModelMessage]:
    """
    Filters out SystemPromptPart from ModelRequest objects

    Args:
        messages: List of ModelMessage objects

    Returns:
        List of ModelMessage objects without SystemPromptPart
    """
    filtered_messages = []

    for msg in messages:
        if isinstance(msg, ModelRequest):
            # Filter out SystemPromptPart
            filtered_parts = [part for part in msg.parts if not
                              isinstance(part, SystemPromptPart)]
            if filtered_parts: # Only add if there are parts left
                filtered_messages.append(ModelRequest(parts=
                    filtered_parts))
        else:
            filtered_messages.append(msg)

    return filtered_messages
```

To address this, we implemented a filtering mechanism that processes the messages retrieved from the *result* object, keeping only *ModelRequest* objects which contain the user query and agent response. This way, we can maintain the pertinent user and assistant exchanges and provide them to the agent as context, allowing the agent to be aware of which information it still requires and avoiding asking the user to enter redundant, duplicate information.

2.6.2 Retrieval Augmented Generation

As mentioned in section 2.2, we implemented Retrieval Augmented Generation in the agentic system, providing the job description generation agent with additional context. Implementing the Retrieval-Augmented Generation (RAG) mechanism within the job description generation system employs a sophisticated vector database architecture that leverages semantic similarity to augment user-provided job requirements with contextually relevant skills. This system operates through the `analyze_requirements` function, which serves as the core component of the RAG pipeline.

The function accepts a `JobRequirements` object - a structured representation of job-related data containing essential, soft, and nice-to-have skill requirements along with position metadata such as job title, seniority level, responsibilities, salary range, location, and employment type. Within the function, a three-stage RAG process unfolds:

1.Vectorization Stage The system systematically processes each requirement by generating dense vector embeddings using OpenAI's `text-embedding-3-small` model. This model maps each textual skill requirement to a high-dimensional vector space (1536 dimensions) where semantic relationships between skills are preserved as geometric relationships. The embedding generation occurs through an API call to OpenAI's embedding service:

```
embedding_response = deps.openai_client.embeddings.create(
    model="text-embedding-3-small",
    input=skill.requirement,
)
skill_embedding = embedding_response.data[0].embedding
```

2.Retrieval Stage For each vectorised requirement, the system queries a Supabase vector database using a Remote Procedure Call (RPC) to the `match_requirements` function. The database has installed the `pgvector` extension (version 0.8.0), enabling efficient similarity searches over high-dimensional vectors.

```
similar_skills = deps.supabase.rpc(
    'match_requirements',
    {
        'query_embedding': skill_embedding,
        'match_count': 2,
        'job_title_filter': requirements.job_title,
        'requirement_type': skill.type
    }
).execute()
```

In this case, the RPC interface accepts the query embedding vector, a limit on the number of matches to return, 2, a job title filter for contextual relevance, and the requirement type (to ensure like-for-like matching). Internally, the Supabase function performs a cosine similarity search against the `vector_table`,

which stores job requirements as JSON data and their corresponding vector embeddings. Each entry in this table contains:

- **job_title**: Text field identifying the position
- **requirement**: JSONB field containing the requirement type and text
- **requirement_embedding**: Vector field storing the pre-computed embedding

3. Augmentation Stage The retrieved similar skills are processed and categorised by type (essential, soft, or nice-to-have). For each retrieved skill, the system extracts the skill data and its similarity score:

```
for result in similar_skills.data:
    data = result['requirement']
    similarity = result['similarity']
    print(f"Similarity: {similarity}")
    if data['type'] == 'skill_requirement':
        final_essential_skills.append({
            'type': data['type'],
            'requirement': data['requirement']
        })
    # Similar code for soft_skill_requirement and
    # nice_to_have_requirement
```

The processed skills are then assembled into an `UpdatedRequirements` object that maintains the original structure but with augmented content. This object is used in the subsequent job description generation phase, where both the original user-provided requirements and the RAG-enhanced requirements are presented to the LLM:

```
generation_prompt = JOB_DESCRIPTION_GENERATION_PROMPT.format(
    job_title=requirements.job_title,
    original_essential_skills=chr(10).join(f'- {skill.requirement}' for
        skill in requirements.job_requirements.essential_skills),
    inclusive_essential_skills=chr(10).join(f'- {skill.requirement}' for
        skill in updated_requirements.essential_skills),
    # Similar formatting for other skill types
    # ...
)
```

The system utilises this dual representation approach to provide the LLM with the user's original intent and contextually relevant ways of wording said requirements. This approach allows the LLM to phrase the user's requirements in a professional, naturally sounding manner.

The database architecture supports this RAG pipeline through a specialised table schema that accommodates textual and vector data, with appropriate indexing to facilitate efficient similarity searches.

2.6.3 Self-Refining Judge Model

The Judge Agent represents a critical component in our job description generation system. It enforces quality standards across multiple dimensions, ensures inclusivity, and drives iterative improvement of generated content. This section details our implementation, design decisions, and technical integration.

Master Prompt The Judge Agent relies on a detailed five-bucket rubric (rationale behind bucket choices in table 1), which is part of its system prompt, the relevant code snippet can be found in the appendix. The prompt provides extensive examples for detecting non-inclusive language patterns, giving the agent both pattern recognition capabilities and explicit remediation strategies:

```
"""
Issue: Using "essential".
Solution: Think about what is actually essential, and what can be taught
...

Issue: Time taken to apply and number of stages in the process.
Solution: Expose the number of steps and think about why this number is
important...

Issues: Years of experience needed.
Solution: Think about the actually skills somebody needs...

Issue: "Flexible working"
Solution: What does this mean? For some people it's true flexibility...

Issue: "Enhanced parental/maternity leave"
Solution: What does this mean? Expose your policy on this...
Refer to parental leave as opposed to maternity leave or paternity leave.

Issue: Listing statutory holiday days as a "benefit"
Solution: Increase the amount of leave allowed by your employees...

Issue example: "5 years experience of SQL"
Solution: Do we need to list the programming language?...

Issue example: "experience working with teams"
Solution: This is vague and doesn't mean anything...
"""
```

This method of providing custom demonstrations to match the problem at hand - referred to as tailor prompting - consistently outperforms generic examples, especially in complex tasks such as diagnosing language as gender biased [63],[64]. It is crucial for these examples to be of high quality, as poor quality ones can decrease the performance of an LLM [65],[66]. To ensure our examples met this standard, we used Dr Alice Ashcroft's (part owner of JobFair) expertise to craft them.

Tool and Framework Orchestration The Judge Agent is implemented using a modern, modular architecture that leverages several specialised frameworks:

- **Pydantic-AI:** Provides the agent definition, type safety, and structured I/O
- **FastAPI:** Handles API endpoints and request validation
- **OpenAI API:** Powers the large language model (GPT-4o) for evaluation
- **textstat:** Performs readability analysis on job descriptions

The refinement flow follows these steps:

1. The job description agent generates an initial draft based on the requirements(slots from the chatbot)
2. The draft is passed to the judge agent via the `evaluate_job_description` tool
3. The judge agent analyses the text using its rubric and readability tool
4. A structured `JudgeResult` is returned with score and improvement items
5. If the score is below the threshold (7/10) or fails critical criteria:
 - (a) The job description agent incorporates all feedback
 - (b) A new draft is generated with improvements
 - (c) The evaluation-refinement cycle repeats until passing criteria are met
6. The final approved job description is returned to the user

Structured Output The Judge Agent's output is formalised using Pydantic models, ensuring type safety and validation, similar to the Requirements gathering agent discussed earlier in this section:

```
class FeedbackItem(BaseModel):
    category: str = Field(description="The evaluation criterion or category")
    item: str = Field(description="Specific area that needs improvement")
    reasoning: str = Field(description="Detailed reasoning behind flagging this area")
    recommendation: str = Field(description="Actionable recommendation for improvement")

class JudgeResult(BaseModel):
    score: float = Field(ge=0, le=10, description="Score out of 10")
    passed: bool = Field(description="Whether the job description meets requirements")
```

```

feedback: str = Field(description="General summary feedback for
improvement")
improvement_items: List[FeedbackItem]

```

Here's an example of the structured output:

```

{
  "score": 6.5,
  "passed": false,
  "feedback": "The job description needs improvements in inclusivity and
    readability.",
  "improvement_items": [
    {
      "category": "Inclusivity",
      "item": "Gendered language",
      "reasoning": "The phrase 'he must have' assumes male candidates.",
      "recommendation": "Replace with gender-neutral 'you must have' or '
        the candidate must have'"
    },
    {
      "category": "Readability",
      "item": "Flesch Reading Ease",
      "reasoning": "Current score of 52.3 is below the minimum threshold
        of 60.",
      "recommendation": "Simplify sentence structure and use shorter
        paragraphs."
    }
  ]
}

```

The detailed feedback from the "judge" informs the agent of where the mistake is located and provides actionable changes that can be directly implemented.

Iterative Refinement Loop The judge is integrated into the job description generation workflow through a dedicated function exposed as a tool. This function is iteratively called until the job description receives a "passing" score.

To offload some of the workload away from the agent, we implemented tools. Tools are often functions which an agent can use to achieve a certain goal. These are often used to solve simple problems programatically rather than relying purely on the LLM. The rationale for this will be discussed in the next section 2.6.3 For readability assessment, we implemented a specialised tool that calculates the Flesch Reading Ease score:

```

@judge_agent.tool_plain
async def evaluate_readability(description: str) -> str:
    """
    Evaluates the Flesch reading ease score of the job description.
    Args:
        description: The complete job description as a string
    """

```

```
score = textstat.flesch_reading_ease(description)
return f"Flesch Reading Ease Score: {score:.1f}"
```

Engineering Decisions Several key engineering decisions shaped the Judge Agent implementation:

- **LLM Selection:** We chose GPT-4o for its performance/cost ratio, although logically it would make sense to have a "smarter" model as the judge, the paper from Jiang et al 2023 [37], repeatedly states that even a small size instruction-fine tuned model can be effective at correcting a larger LLM. However, if future models show significant performance improvement, the utilisation of Pydanticai as an agent orchestration framework allows us to change the model used easily.
- **Hybrid Analysis:** As noted in many papers, LLM struggle with mathematical/calculation-based tasks [67],[68]. To combat this, we implemented "tools" such as the `evaluate_readability` function, which allows the agent to hand off the burden of calculating metrics.
- **Structured FeedbackItems:** Rather than providing free-form feedback, we structured the output with specific categories, items, reasoning, and actionable recommendations—facilitating targeted improvements.
- **Pass/Fail Criteria:** We implemented explicit thresholds (7/10 overall, minimum 1.5/2 for inclusivity, 60+ for Flesch readability) to enforce minimum quality standards.
- **Memory-Augmented Refinement:** By instructing the job description agent to incorporate feedback cumulatively through the `message_history` mechanism, we prevent regression and ensure continuous improvement.

Limitations and Future Work Despite our robust implementation, several limitations remain. The Judge Agent's evaluation is inherently subjective and may exhibit inconsistent scoring across similar inputs. The logic behind the pass threshold is also not empirically validated as optimal; we did internal testing and concluded that 7/10 provided a good balance between quality and speed. Additionally, the iterative refinement loop lacks a hard cap on iterations, potentially leading to excessive API calls or diminishing returns on minor improvements.

A promising enhancement would be integrating a skills-ontology plugin that validates technical skills against industry databases. Although we have already done this during the pre-generation phase by utilising RAG, our database is limited to only 400 job descriptions, which lacks a wide enough variety to account for all edge cases. This would ensure skill requirements are current, relevant, and properly categorised by domain and seniority level. Such a plugin could catch deprecated technologies or inconsistent skill combinations that the current system might miss, further improving job description quality and relevance.

2.6.4 Editing and Version History

Effective job description editing requires maintaining context and allowing users to reference or revert to previous iterations. The version history system addresses this need by systematically storing and managing different states of the job description throughout the editing process.

Purpose and Rationale The primary goal of the version history feature is to provide a robust mechanism for tracking changes, facilitating comparative analysis between different versions, and enabling users to leverage past content during revisions. This supports a non-linear editing workflow where users can experiment with changes, review historical states, and build upon previous iterations without losing work. It directly contributes to the project objective of creating a flexible and user-centric job description editing environment.

Agent Integration and Prompting The ‘editor_agent’ is explicitly instructed via its system prompt to utilise the version history stored within the conversational context (‘message_history’).

```
# EDITOR_AGENT_PROMPT snippet
"""
...
3. Version Management:
- all previous job descriptions produced are in the ModelResponse object
  within your message history context
- Reference specific versions when discussing changes
- If a user asks you to provide an earlier version of the job_description
  use your message history context
- The dynamic prompt will provide you with the current version of the
  job description, so use that when filling out the JobDescription
  version field
EXAMPLE:
User: Can you use version 2 of the job description to help me write the
      new one?
Assistant: Retrieve the JobDescription from the message history context
           with version number 2.
- Maintain context across revisions
CRITICAL: Unless specified otherwise by the user, assume you are altering
           the
           job description provided by the user in the user prompt.
...
For the version field, ensure you return the same version as the provided
           job
           description, i.e do not alter the version field.
           e.g if the user job description has version 1, your output job
           description
           should also have version 1.
Always return a complete JobDescription object with your changes...
"""
```

This prompt guides the agent in retrieving specific prior versions when requested and maintaining the correct version number when applying edits to an existing version. We leveraged the use of the keyword *"CRITICAL"* to ensure the agent only edits a previous version when explicitly specified by the user.

API Endpoints and Workflow Several FastAPI endpoints facilitate version management:

- **POST /edit_chat:** Handles general editing requests. It retrieves the relevant 'Thread', passes the 'edit_model_history' to the 'editor_agent', and processes the resulting 'JobDescription'. It uses the version provided in the request's 'current_job_description' for context and maintains this version number in the response unless a new version is explicitly created.
- **POST /create_new_version:** Explicitly creates a new version snapshot. It takes the 'current_job_description', assigns it a new sequential version number (based on the current count in 'thread.descriptions'), and appends the new 'EditedJobDescription' to the 'thread.descriptions' list.
- **POST /save_version:** Updates an **existing** version in the 'thread.descriptions' list. It finds the description matching the provided version number and replaces it with the updated content and recommendations.
- **GET /api/threads/{thread_id}/version-histories:** Retrieves all stored versions ('EditedJobDescription' objects) for a specific thread, allowing the frontend to display the historical progression.

The typical workflow involves:

1. User initiates an edit via '/edit_chat' on the current version.
2. The 'editor_agent' uses its 'message_history' (containing past interactions and potentially prior 'JobDescription' objects) and the 'current_job_description' provided in the prompt to perform the edit.
3. The updated 'JobDescription' (maintaining the **same** version number) is returned.
4. Optionally, the user saves this as a **new** distinct version via '/create_new_version', incrementing the version count.
5. The user can view all saved versions via '/api/threads/thread_id/version-histories'.

Limitations and Future Work The current implementation relies on an in-memory dictionary ('threads') for storing version history, which is not persistent and limits scalability. Storing full 'ModelMessage' history can also become memory-intensive for long conversations. Retrieval complexity for the agent might increase with very long histories.

Future work could involve:

- Implementing persistent storage (e.g., using the existing Supabase integration) for ‘Thread’ data, including the ‘descriptions’ and ‘edit_model_history’.
- Introducing summarisation or pruning strategies for ‘edit_model_history’ to manage context length for the LLM while preserving essential version information.
- Developing a more sophisticated retrieval mechanism for the agent, potentially using vector embeddings of past versions to find relevant historical context more efficiently than linear scanning of message history.

2.7 Rapid Prototyping

Once the agentic back-end was constructed, we needed to develop a front-end interface that ties together the creation of a job description through a chatbot interface with streamlined editing. As mentioned in section 1.3, one of the key problems with the old MVP product was the lack of in-app editing. Users would have to switch to a different editing app, change their job description, and re-upload it for analysis. This fragments their work sphere and is a bad design choice, as the app inhibits uninterrupted creative flow. This interface fragmentation has been shown to degrade productivity and user satisfaction across domains. To resolve this, Cycle 2 built upon the agentic system by creating a Streamlit app that unifies generation and editing in one workplace. The following section will discuss the design rationale behind choosing Streamlit for prototype development, followed by a section explaining the implementation.

2.7.1 Design and Rationale

In choosing a rapid prototyping framework for Cycle 2, our goal was to quickly validate the integrated “generate + edit” workflow, gather early stakeholder feedback, and avoid the sunk-cost risks of building a full-scale front end before verifying backend behaviour. Low-fidelity rapid prototyping has been shown to swiftly and cheaply identify major interface flaws, reducing overall development time by up to 50% and avoiding rework costs later in the cycle [69]. To further ensure this prototype could be developed quickly, we decided to go with a low-code platform that provides pre-built modules and visual components, ensuring all of our efforts could go towards evaluating the performance of the agentic system. Using such frameworks has been shown to accelerate internal solution delivery [70]. The framework we chose for this prototype was Streamlit. However, there were other alternatives considered:

1. **Dash:** Dash provides a React-based, callback-driven architecture suited to complex, multi-page enterprise dashboards. It offers granular control over layout, theming, and inter-component communication, but at the cost of boilerplate: every UI element and callback must be explicitly defined

2. **Panel:** Panel (part of HoloViz) integrates tightly with the PyData stack, supports both high- and low-level APIs, and can serve Bokeh, Matplotlib, or Plotly charts alongside DataFrame tables. Although its flexibility is compelling, its initial setup and callback wiring introduce significant overhead, making it more suitable for complex analytics apps.
3. **Anvil:** Anvil provides a full drag-and-drop IDE coupled with Python back-end code hosting, enabling non-web developers to build complete apps without JS/HTML. While powerful, it introduces platform lock-in and requires learning Anvil’s proprietary component model.

Why Streamlit?

1. **Zero Front-end Context Switch:** Streamlit lets you write both UI and hook into your Python agentic back-end in one file—no separate React/JSX knowledge required
2. **Rapid Iteration with Hot-Reload:** Changes to a Streamlit script reflect instantly in the browser, enabling same-day proof-of-concept demos and fast stakeholder feedback
3. **Built in state management:** Managing *st.session_state* avoids manual workarounds or additional libraries, keeping slot-filling chatbot inputs and incremental edits persistent across interactions
4. **Minimal Setup&Development:** A single pip install streamlit and streamlit run app.py is all that’s needed to show stakeholders a working POC, versus multi-package configs or proprietary platforms
5. **Sufficient UI richness:** Streamlit’s widgets—text areas, buttons, sliders, and markdown—cover the needs of an inclusive-language editing tool without the overhead of enterprise component libraries

By weighing development speed, ease of integration, and cognitive overhead, Streamlit emerges as the optimal Python framework to rapidly validate and demonstrate the integrated “generate + edit” workflow in Cycle 2.

2.7.2 Implementation of Streamlit Prototype

The implementation follows a stateful, event-driven architecture to manage the conversational flow between the requirements gathering and job description editing phases.

Requirements Gathering Phase

During the initial phase, the system presents a sequential series of questions to elicit job requirements. Input collection occurs through a chat interface that processes natural language responses:

```
if not st.session_state.requirements_gathered:
    user_input = st.chat_input("Your response:")
```

```

if user_input:
    result = await chat(user_input, st.session_state.messages, deps)
    # Processing logic

```

This phase continues until the system has accumulated sufficient information to generate an initial job description, at which point the `requirements_gathered` flag triggers a transition to the editing phase.

Job Description Editing Phase

Once the initial job description is generated, the interface transforms to present both the current job description and a specialised editing input:

```

if st.session_state.requirements_gathered and st.session_state.
    job_description:
        st.markdown("### Current Job Description")
        st.write(st.session_state.job_description)

    # Display edit message history
    for message in st.session_state.user_messages:
        # Display code

    edit_input = st.chat_input(f"Request changes to the following job
                               description:")

```

This phase employs a different conversation model focused on refinement, allowing users to:

1. View the current job description in its entirety
2. Request specific modifications through natural language instructions
3. See a history of their edit requests and the resulting changes
4. Implicitly build a version history as modifications are applied

State Transitions and Data Flow

The successful collection of structured requirements data determines the transition between phases:

```

if result.data and isinstance(result.data, JobRequirements):
    st.session_state.requirements_gathered = True
    requirements = result.data

    # Get updated requirements from vector search
    updated_requirements = await analyze_requirements(deps, requirements)

    # Generate job description
    # ...

    st.session_state.job_description = description_result.data.
        description

```


When requirements are successfully structured into a `JobRequirements` object, the system:

1. Updates the state flag to trigger the UI transition
2. Initiates the RAG process through the `analyze_requirements` function
3. Generates the initial job description using both original and RAG-augmented requirements
4. Updates the session state with the generated description
5. Persists the job description to the Supabase backend for future reference

This implementation design creates a seamless, guided experience that progressively transitions from information gathering to collaborative editing. It maintains context throughout the process while presenting an appropriate interface for each phase of the workflow.

Key Takeaways This cycle aimed to develop a simple interface with which we could test the new workflow. Our internal testing concluded with the following:

1. Chatbot workflow for requirements gathering was preferred by members of the team
2. Generate job description was of higher quality
3. Only using the agent for edits could be frustrating, as it struggled to change specific parts of the text.
4. The CEO wanted to allow users to make edits, and then choose whether or not to apply the suggestions generated by the gender expert agent.

2.8 Front End and UI

After concluding the internal test of the system using the prototype streamlit interface, we could commit to building a more fleshed-out interface for user testing. Our findings from internal testing were that the pre-job generation phase, i.e requirements gathering chatbot and the pipeline for producing a job description, were satisfactory. However, the post-editing could have improved from a user experience point of view. The library used for this was React, and we decided to mimic a "canvas" interface for post-job generation editing. The React canvas constitutes Cycle 3 of our Design-Science Research programme and maps to ISO 9241-210 activities "produce design solutions" and "evaluate against requirements". It aims to operationalise the user-centred requirements gathered in Cycles 1–2 by (a) eliminating context-switch costs and (b) exposing agentic reasoning in situ for transparency. The following sub-sections explain how each UI element delivers on those goals.

2.8.1 Design and Rationale

Why React When choosing a front-end framework, we had the following points to consider:

1. **Learning Curve and Self Support:** As I was developing this entirely by myself, with no previous front-end experience, the framework needed to be relatively easy to learn, and to have a lot of online support
2. **Future Hiring and Maintainability:** JobFair may be looking to expand its team in future, so it was important for the framework to be mainstream, making the hiring process easier.
3. **Ease of Deployment:** Since this is an MVP product, we need to make changes and easily redeploy the website.

With that in mind, several frameworks were considered but were not sufficient in meeting the objectives stated above:

Table 3: Alternative front-end frameworks and their deal-breakers

Framework	Strengths	Deal-breakers for this project
Angular (16+)	Batteries-included; powerful CLI	Steep learning curve (RxJS, DI, Ng-Modules); heavier runtime; smaller talent pool.
Vue 3 / Nuxt	Gentle syntax; good docs	Ecosystem roughly one-third of React; split APIs (Options vs Composition); fewer enterprise widgets.
Svelte / SvelteKit	Near-zero boilerplate; compile-time optimised	Small community; scarce ready-made components (rich-text, analytics); hiring risk.
SolidJS, Qwik, Astro	Ultra-fast hydration; modern DX	Cutting-edge—docs/tooling change monthly; thin ecosystem; little recruiter recognition.
Vanilla JS + Web Components	No framework lock-in; tiny payload	Must hand-roll routing, state-sync, hot-reload, accessibility—too slow for a solo MVP.

With that in mind, React was chosen as it ticks off each one of the requirements stated above:

1. **Learning curve and self-support:** React benefits from the largest ecosystem of free learning resources—over 36 million weekly downloads on npm[71] and 235,000 stars on GitHub[72]—ensuring abundant tutorials, Q&As, and example code.

2. **Future hiring and maintainability:** React is the most in-demand front-end framework, appearing in approximately 52 % of over 250,000 job postings[73], which simplifies recruitment and onboarding.
3. **Ease of deployment:** Next.js (the React meta-framework) provides zero-config, file-system routing out of the box[74] and one-command Vercel deployments that complete in under a minute[75], enabling rapid iterative releases.

Why The Canvas UI After the streamlit prototype, we identified several pain points from the user’s perspective:

1. Users enjoyed having the suggestions displayed, which was present in the old MVP but was removed in the Streamlit prototype
2. We needed explicit version control
3. We needed to have inline editing
4. The agent would sometimes be too liberal with the changes it made

The split canvas UI naturally maps all of these functionalities onto a single coherent page, an alternative was to have several different pages, e.g a version control page, a metrics page and a an editor page, however this would still cause fragmentation of the user’s workflow as they would have to switch tasks [40]. Having the chat canvas layout with three strips encapsulating all editing functionality into one page, we turn what would have been an *interruption* into a glancable side interaction, maintaining flow. The split-canvas layout does more than collapse three tools onto one screen; prior research shows that *blending natural-language interaction with persistent, direct-manipulation widgets outperforms a pure chat stream*. *DynaVis* (CHI 2024) let users edit visualisations through both language and auto-synthesised UI controls; in a 24-person study, participants *preferred DynaVis to an NLI-only interface* because the on-canvas widgets made subsequent tweaks easier and increased confidence via immediate visual feedback [76]. Likewise, the *Collaborative Canvas* probe (IUI Workshops 2024) showed that teams valued AI-generated sticky notes but wanted a shared canvas so they could *review and filter* suggestions before exposing them to the group [77]. The success and findings of these projects support our canvas design as a preferable way of collaborating with LLM. The remainder of this design section will justify design choices for each of the strips shown in *add figure*

Strip-1 -Chat to Edit Pane This is the main pane through which the user can utilise the LLM for editing. The rationale behind including an LLM for editing is simple; these models effectively edit structured and semi-structured documents using only simple prompts [78]. Although the Weber study found the LLM’s performance on editing tasks to be linked with how well the document was structured, this wasn’t a concern, as the job description from the agentic pipeline already produced a well-structured document with headings.

In particular, utilising LLM for document editing has been shown to boost productivity by reducing manual effort from the users’ side [79]. This design choice contributes to solving the previously identified problem of the MVP lacking seamless in-context editor functionality.

We made a noticeable change to this AI editor agent by changing the system prompt from the Streamlit Prototype. Previously, the agent would take in a user query and automatically make changes using gender neutral language. However, as stated in section 2.7.2, after internal testing of Cycle 2, the CTO of the company said he preferred allowing users to make the exact changes they wanted, and to be given the option to make these changes gender neutral. This provides the user with even more fine-grained control over which changes they wish to apply.

Strip-2 Canvas Editing This strip is where the currently selected job description is displayed. The canvas supports an edit mode allowing the user to edit the job description freely. The rationale behind this functionality was based on the Collaborative Canvas study [77], which concluded that although users enjoyed the assistance of LLMs, they wanted to have the freedom to review and make changes to the generated output manually. While in editing mode, the user can select text for "AI editing"; this mode allows for selecting specific text which the edit agent should change. This provides a more fine-grained method of editing in addition to using the LLM agent in the chat window, which will try to make changes. However, as noted in the reflection section of the streamlit prototype, we found that the agent was sometimes too liberal with changing text. A pure chat prompt asks the LLM to "guess" the edit span; it therefore tends to rewrite more text than the user intended—a problem we observed in the Streamlit pilot during internal testing. Highlight-to-edit removes that ambiguity by binding the selection range to the agent request, which delivers two advantages backed by recent studies:

1. InkSync’s two controlled studies showed that interfaces which bind executable edits to a user’s text selection lead to significantly faster revisions and fewer factual errors than chat-only editing [80]
2. In the collaborative canvas user reported that they trusted the AI suggestions only after they could "pinpoint exactly where the changes would go" and keep the rest of the document intact [77] Our select-to-edit mode mirrors that affordance, letting recruiters keep control of tone while still leveraging the agent for micro-edits.

By embedding this highlight workflow directly in the canvas, we give users a *precision scalpel* instead of a *broad brush*, correcting the over-editing pain surfaced in the prototype and aligning with evidence that selection-scoped commands improve both accuracy and user confidence.

The last feature we implemented was changing the version control system used in the Streamlit Prototype. In the Streamlit prototype, every agent edit

spawned a new revision, quickly flooding the history and obscuring which snapshots mattered. In the React canvas, we switched to an explicit “save → branch” model, like Git: edits remain ephemeral until the user clicks Save, and a Create Version button checkpoints the current state under a new version ID. This puts authorship back in the recruiter’s hands and yields two documented benefits:

1. **Meaningful landmarks:** Creative-work studies show that practitioners prefer to checkpoint at self-defined “conceptual milestones”, not after every micro-change; granular logs degrade recall and increase retrieval time [53]
2. **Safer exploration:** When versions are user-initiated, recruiters can run bold AI edits, review the result, and abandon it without polluting the timeline—an interaction pattern that Chan & Schunn found to encourage risk-taking and ultimately higher-quality outcomes in iterative tasks [54]

Strip-3 Version– Metrics Strip The right-hand strip couples two elements—the version timeline and the live-metrics panel—to support reflection in action without forcing a context switch.

Version timeline. Each commit appears as a collapsible card labelled “v1”, “v2”, Expanding a card reveals the inclusive-language suggestions generated by the agentic judge and a short rationale (e.g. “Avoids masculine pronoun”). Clicking a suggestion highlights the affected span on the canvas, allowing the recruiter to inspect or accept it. Providing the explanatory snippet preserves JobFair’s pedagogy goal: concise “why/how” explanations have been shown to raise user trust and perceived usefulness compared with black-box corrections[3]. Commits are created only when the user presses *Create Version*, meaning the timeline contains meaningful milestones rather than a flood of auto-saves as mentioned in section 2.8.1

Live-metrics panel. Beneath the timeline, a small dashboard streams the current Flesch Reading Ease and word count as the recruiter types. Instant, surface-level feedback nudges authors toward clearer, shorter prose: texts above FRE 60 are consistently rated easier to comprehend by broad audiences[?]. Displaying the scores in real time turns readability from a late validation step into a continuous guidance cue, letting recruiters converge on an accessible, inclusive draft without leaving the canvas.

2.9 Implementation (React / Next.js Canvas UI)

This section details the front-end implementation of the three-strip canvas user interface using React and the Next.js framework, realising the design goals of minimising context switching and exposing agentic reasoning transparently.

ARCHITECTURE & FILE STRUCTURE For this final cycle, JobFair’s designer provided me with the following designs in Figma as shown by figure 10

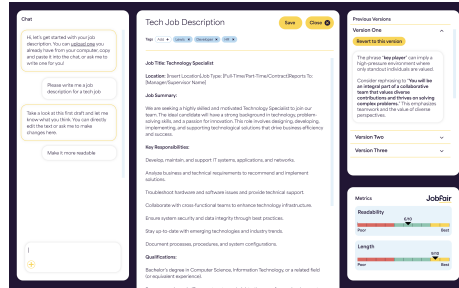


Figure 10: Figma Design of React UI

The app was logically divided into three strips, each of which housed specific functionality. For clarity, Figure ?? shows a higher-level view of the architecture for this UI.

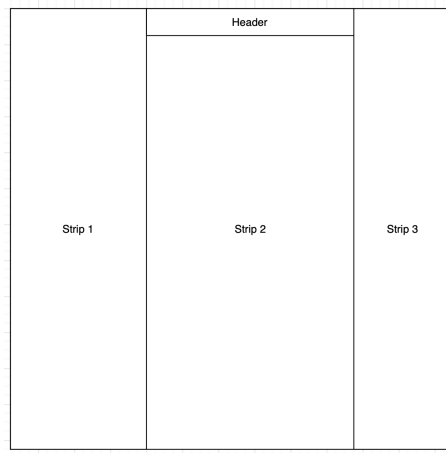


Figure 11: High Level UI diagram

- **Strip 1:** Contains the chatbot functionality for editing, users can make requests through an open dialogue with the LLM. This functionality is the "edit agent" from the streamlit cycle and is the main way the user can interact with the job description
- **Strip 2:** This strip is further divided into two parts:
 - **Header** The Header contains a title for the job description and houses the "save" and "edit with AI" buttons. Reflecting the design decision to move away from automatic versioning per edit, the

front end implements an explicit "Save → Branch" model. User edits on the canvas are ephemeral until committed. The **Create Version** button triggers a call to the `src/services/createNewVersion.js` service. This function packages the current job description state (title, description text, current version number, and associated gender recommendations) and sends it to the backend `/create_new_version` endpoint. Approach ensures the version history (`VersionHistory.js`) reflects meaningful, user-validated milestones rather than every intermediate agent suggestion or minor tweak, directly addressing feedback from prototype testing and aligning with research on creative workflows [53]. When clicked, the "edit with AI" button allows the user to highlight text that can be used in conjunction with the chat pane located in strip 1 to make very localised changes to the text. This was implemented using a separate editing agent which has a different system prompt:

Listing 1: User Prompt for Applying Gender Neutral Change

```

APPLY_CHANGE_USER_PROMPT = ""
Please change the following gendered item to a
gender neutral item:{gendered_language} in
accordance to the following reasoning:{reasoning}
}
The current job description is as follows:
{job_description}
Please return the updated job description in the
same format as the original job description.
""

```

The rationale for creating this separate agent was that the edit agent uses message history for context, while this selection-based editing task doesn't require such contextual information, as its intended use is to make small but precise changes to the job description.

- **Job Description:** This pane contains the currently selected job description, and is the focal point of the interface. As mentioned previously, one of the features we included was highlighting non-inclusive language. For this, we used a Regex pattern matching function, as shown by listing 2

Listing 2: HighlightedJobDescription React Component Snippet

```

function HighlightedJobDescription({ text,
  highlightPhrases = [] }) {
  // If no phrases to highlight or no text, return
  plain text
  if (!highlightPhrases.length || !text) {
    return <p className={styles.description}>{text}</
    p>;
  }
}

```

```
// Escape special regex characters
const escapedPhrases = highlightPhrases.map(phrase
=>
  phrase.replace(/[*+?${}()|[\]\]/g, '\\$&')
);
// Create regex pattern for all phrases
const pattern = new RegExp(`(${escapedPhrases.join(
  '|')})`, 'gi');

// Split text into segments (alternating normal and
  highlighted)
const segments = text.split(pattern);
```

The function takes a variable text which encompasses the entire job description. The variable highlighted phrases are all the gendered items picked up by the edit agent during changes.

- The last strip contains the version items and metrics menu. The versions are all stored in a collapsible box. The current job description is updated by clicking on one of these, and all of the suggestions are shown to the user. Figures .. and .. show this functionality. Strip 3 also houses the metrics menu. Each metric was constructed to be a reusable component called Metric.

Listing 3: Metric Component Usage in JSX

```
<Metric
  title="Readability"
  score={readabilityScoreRounded}
  maxScore={10}
/>
```

As we can see from 3, each metric takes three props, a title, score and a maxScore. This allows JobFair to swap out metrics shown to the user easily.

STATE MANAGEMENT & CONTEXT PROPAGATION The application-level state, such as the current job description text, chat history, version list, and computed metrics, needs to be accessible across multiple components within the three-strip layout. Given the moderate complexity and the need to avoid premature optimisation for this Minimum Viable Product (MVP), we opted for React's built-in Context API for managing shared state related to versioning, as seen in `src/app/generated/VersionContext.js`.

Listing 4: Version Context Provider Snippet (`src/app/generated/VersionContext.js`)

```
"use client";
import { createContext, useContext, useState } from 'react';
```



```

const VersionContext = createContext();
export function VersionProvider({ children, initialVersions = [],
  setDescription }) {
  const [expandedVersion, setExpandedVersion] = useState(null);
  return (
    <VersionContext.Provider value={{ expandedVersion, /* ... */ versions
      : initialVersions }}>
      {children}
    </VersionContext.Provider>
  );
}

export function useVersionContext() { return useContext(VersionContext);
}

```

This provider is wrapped around relevant parts of the component tree, making version data and interaction handlers available via the `useVersionContext` hook. While effective for the current scale, more complex state interactions or future needs like fine-grained performance optimisation or time-travel debugging might warrant migration to a dedicated library like Zustand or Redux. Zustand was considered for its minimalistic API and performance benefits, but it was deemed unnecessary dependency addition at this stage. Other state elements (like chat history or the current canvas content) are managed via component state (`'useState'`).

REFLECTION The current React/Next.js implementation successfully translates the three-strip canvas design into a functional interface. The adherence to Next.js conventions provides a solid foundation. The choice of Context API for version state strikes a balance between capability and simplicity for the MVP stage, though scaling might necessitate revisiting state management with libraries like Zustand. The service layer effectively isolates backend communication, but could be enhanced with more robust error handling, loading state management visible to the user, and request cancellation logic using `AbortController`. The reusable `Metric.js` component exemplifies good practice for future extensibility. Overall, the implementation meets the requirements set out at the end of Cycle 2 and provides a robust platform for user testing and future feature development.

3 Evaluation

3.1 Introduction

3.1.1 Purpose of Evaluation

This evaluation aims to empirically assess the redesigned chatbot-driven workflow (Workflow 2), which integrates the agentic backend and React canvas UI

developed in this project. Its performance and user experience will be rigorously compared against the original Minimum Viable Product (MVP) system (Workflow 1) to determine the effectiveness of the implemented improvements in addressing the limitations identified in Section 1.2.

3.1.2 Evaluation Goals

The specific goals for this evaluation are:

- To compare the *usability* of Workflow 1 and Workflow 2 for generating a job description.
- To compare the *perceived usefulness* of Workflow 1 and Workflow 2 for generating inclusive and complete job descriptions.
- To compare the *efficiency* (time taken) of generating a job description using Workflow 1 versus Workflow 2.
- To compare the *quality* (including inclusivity, completeness, clarity, accuracy, and engagement based on the Judge Agent rubric of job descriptions generated by Workflow 1 versus Workflow 2.
- To evaluate the *perceived usefulness* of the newly implemented editing functionalities (in-context canvas editing, AI assistance, version control) within Workflow 2.
- To gather qualitative insights into the user experience of the integrated editing features in Workflow 2.

3.1.3 DSR & ISO compliance

This evaluation is grounded in the Design Science Research (DSR) paradigm adopted for this project, which mandates rigorous assessment of the designed artefact *need to cite DSR*. Furthermore, the evaluation structure and chosen metrics align with the ISO 9241-210 *need to cite ISO* standard for human-centred design, focusing on assessing the system’s effectiveness, efficiency, and user satisfaction within the intended context of use.

3.1.4 Research Questions

This evaluation seeks to answer the following specific research questions:

- RQ1:** Does the new system make the job description generation easier than the old one (SUS)?
- RQ2:** Is the new system subjectively better at performing the task of creating an inclusive job description (Perceived Usefulness Questionnaire - Generation)?

RQ3: Is the new system better at generating higher quality job descriptions (Evaluation with judge)?

RQ4: Does the new workflow’s editing functionality aid the recruiter’s in refining a job description effectively (Perceived Usefulness questionnaire - Editing)?

Additionally, qualitative feedback will be gathered via open-ended questions to understand the nuances of the user experience regarding the editing features of Workflow 2.

3.2 Evaluation Design & Methodology

3.2.1 Overall Approach

A mixed-methods approach was employed. A within-subjects experimental design was used to primarily compare Workflow 1 and Workflow 2, focusing on the job description generation task. This allowed each participant to act as their own control, reducing inter-participant variability. Following the comparative task, an exploratory phase explicitly focused on the editing features of Workflow 2, evaluated using a Perceived Usefulness scale adapted from the TAM model and brief qualitative feedback collection.

3.2.2 Participants

The target participants for this study are recruiters or individuals involved in the hiring process who have experience creating job descriptions. Recruitment will be attempted via Josh Bailey’s network of recruiters in Fraser House Lancaster. *(Note: Final participant details depend on ethics approval and recruitment success via Fraser House contacts, coordinated by Josh.)*

3.2.3 Procedure

Each participant session followed a structured procedure:

- **Introduction and Consent:** Participants received an information sheet outlining the study’s purpose, procedure, data handling (anonymisation, secure storage), and their rights, including the right to withdraw. Written informed consent was obtained. A brief demographic questionnaire was administered.
- **Scenario Briefing:** Participants chose an occupation of their choice, from which they generated a job description using bot workflows. By not enforcing a specified role across all users, we allowed them to use the system more organically and realistically.
- **Part 1: Comparative Generation Task:**

- Participants used Workflow 1 (the original MVP) to generate a job description based on the scenario. The task completion time was recorded from when they started interacting with the system until the generation was completed.
- Participants then used Workflow 2 (the redesigned system) to generate a job description for the same scenario. Task completion time was recorded similarly.
- The order in which participants used Workflow 1 and Workflow 2 was counterbalanced across participants to mitigate potential learning or fatigue effects.
- **Post-Generation Surveys:** Immediately following the completion of both generation tasks, participants completed:
 - The System Usability Scale (SUS) questionnaire, evaluating both systems (administered via separate forms distinguishing the workflows).
 - The Perceived Usefulness (Generation) survey, evaluating the usefulness of both systems for the generation task.
- **Part 2: Exploratory Editing Task (Workflow 2 Only):**
 - Participants were allocated a fixed period of 5 minutes to explore the editing functionalities of Workflow 2, using the job description they had just generated.
 - They were given brief explanation of how the editing features worked.
- **Post-Editing Survey:** Following the exploratory editing task, participants completed the Perceived Usefulness (Editing) survey, focusing on their experience with Workflow 2's editing features.
- **Part 3: Qualitative Feedback Collection:** At the end of the session, participants were asked targeted open-ended questions regarding their experience with the editing features: "Could you share your thoughts on using the editing features in the new system? What worked well or not so well?" and "Overall, how did having these editing tools integrated affect your process?". The researcher then recorded their comments.
- **Debriefing:** Participants were thanked, reminded of the study's purpose, and allowed to ask any final questions.

3.2.4 Data Collection Instruments & Metrics

Data was collected using the following instruments and metrics, which are mapped to the ISO 9241-210 goals of Effectiveness, Efficiency, and Satisfaction:

Effectiveness Measures: Primarily assessed via the quality of the generated job description output.

- *Output Quality (Judge Agent Score):* Job descriptions generated using both Workflow 1 and Workflow 2 during the participant tests were collected. These outputs were evaluated asynchronously by the custom Judge Agent, developed based on Alice Ashcroft’s framework. This yields a quantitative score (e.g., 0-10) reflecting the description’s quality across criteria like inclusivity, clarity, completeness, etc.

Efficiency Measures: Assessed via the time taken to generate a job description.

- *Time on Task (Generation):* Recorded in seconds (or minutes) from the start of interaction with the scenario brief until the participant indicated the generation task was complete for each workflow.

Satisfaction Measures: Assessed via user’s subjective responses during generation.

- *System Usability Scale (SUS):* The standard 10-item SUS questionnaire was used to measure perceived usability, yielding a score from 0-100 for each workflow.
- *Perceived Usefulness (Generation):* An adapted version of the Perceived Usefulness scale from the Technology Acceptance Model (TAM) was used. This survey, employing six items on a 7-point Likert scale: Strongly Disagree to Strongly Agree, measured the perceived usefulness of each workflow specifically for the *generation* task. Items were adapted by naming the respective workflow (e.g., "Workflow 1 (MVP)", "Workflow 2 (New System)").

Editing Feature Evaluation Measures (Workflow 2 Only): Focused on the novel editing capabilities, primarily assessing Satisfaction.

- *Perceived Usefulness (Editing):* A similarly adapted TAM Perceived Usefulness survey was administered after the exploratory editing task, focusing specifically on the usefulness of Workflow 2’s *editing* features (e.g., canvas editor, AI suggestions, versioning).
- *Qualitative Feedback Comments:* Participant comments, recorded in response to the open-ended questions asked at the end of the session regarding their experience with the editing features.

3.3 Results

3.4 Key Takeaways

4 Conclusion

5 Appendix

```
JUDGE_AGENT_PROMPT = """
You are an expert job description evaluator. Your task is to grade job
descriptions
on multiple criteria. Please use the following detailed rubric to award
points...

1. Clarity & Structure (Max: 2 points)
  - Clear organization (0.5 point): Must have headers such as 'About Us
    ', 'Role'...
  - Logical flow (0.5 point): Content should progress naturally...
  - Mobile-friendly formatting (0.5 point): Use bullet lists with 7
    items per list.
  - Consistency of formatting (0.5 point): All sections are uniformly
    formatted.

2. Technical Accuracy (Max: 2 points)
  - Role-specific skills (0.7 point): Skills and technical requirements
    match...
  - Technology stack specificity (0.7 point): Specific tools and
    frameworks...
  - Certifications justified (0.6 point): Only include certifications
    that are...

3. Inclusivity (Max: 2 points)
  - Language use (1.2): Pick out 1-10 items of language this job
    description...
  - Structural inclusivity (0.8 point): Flesch-Kincaid grade between 8
    and 10...

4. Completeness (Max: 2 points)
  - All key sections present: Includes sections for DEI, growth
    opportunities...

5. Engagement (Max: 2 points)
  - Compelling 'About Us' section (0.7 point).
  - Growth narrative (0.7 point): Clearly describes opportunities like
    training...
  - Authentic tone (0.6 point): Avoid corporate jargon...

CRITICAL:
-THE INCLUSIVITY MUST BE AT LEAST A 1.5 FOR A PASS

```

References

- [1] D. A. Collier and C. Zhang, “Can we reduce bias in the recruiting process and diversify pools of candidates by using different types of words in job descriptions,” 2016.
- [2] T. Romano, F. Mohammadi, and P. Ceravolo, “Synthetic data for identifying inclusive language (case study: Job descriptions in italian),” *2024 IEEE International Conference on Cyber Security and Resilience (CSR)*, pp. 737–742, 2024.
- [3] A. Kartikeya, “Examining correlation between trust and transparency with explainable artificial intelligence,” 2021.
- [4] M. Guesmi, M. A. Chatti, S. Joarder, Q. U. Ain, C. Siepmann, H. Ghanbarzadeh, and R. Alatrash, “Justification vs. transparency: Why and how visual explanations in a scientific literature recommender system,” 2023.
- [5] G. Mark, V. M. Gonzalez, and J. Harris, “No task left behind? examining the nature of fragmented work,” in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, CHI ’05, (New York, NY, USA), p. 321–330, Association for Computing Machinery, 2005.
- [6] H. Iso, P. Pezeshkpour, N. Bhutani, and E. Hruschka, “Evaluating bias in llms for job-resume matching: Gender, race, and education,” 2025.
- [7] K. Chang, S. Xu, C. Wang, Y. Luo, T. Xiao, and J. Zhu, “Efficient prompting methods for large language models: A survey,” *ArXiv*, vol. abs/2404.01077, 2024.
- [8] A. Razavi, M. Soltangheis, N. Arabzadeh, S. Salamat, M. Zihayat, and E. Bagheri, “Benchmarking prompt sensitivity in large language models,” 2025.
- [9] D. Deming and L. B. Kahn, “Skill requirements across firms and labor markets: Evidence from job postings for professionals,” Working Paper 23328, National Bureau of Economic Research, April 2017.
- [10] D. Gaucher, J. Friesen, and A. C. Kay, “Evidence that gendered wording in job advertisements exists and sustains gender inequality,” *Journal of Personality and Social Psychology*, vol. 101, no. 1, pp. 109–128, 2011.
- [11] H. Mihaljević, I. Müller, K. Dill, and A. Yollu-Tok, “Towards gender-inclusive job postings: A data-driven comparison of augmented writing technologies,” *PLOS ONE*, vol. 17, p. e0274312, Sept. 2022.

- [12] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design science in information systems research,” *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.
- [13] “ISO 9241-210:2019 ergonomics of human-system interaction – part210: Human-centred design for interactive systems,” 2019. Accessed 17 April 2025.
- [14] M. Galesic and M. Bošnjak, “Effects of questionnaire length on participation and indicators of response quality in a web survey,” *Public Opinion Quarterly*, vol. 73, pp. 349–360, 2009.
- [15] J. Zhuo, S. Zhang, X. Fang, H. Duan, D. Lin, and K. Chen, “Prosa: Assessing and understanding the prompt sensitivity of llms,” *ArXiv*, vol. abs/2410.12405, 2024.
- [16] M. Rana, K. Hacioglu, S. Gopalan, and M. Boothalingam, “Zero-shot slot filling in the age of llms for dialogue systems,” *ArXiv*, vol. abs/2411.18980, 2024.
- [17] T. Rietz and A. Maedche, “Ladderbot: A requirements self-elicitation system,” 09 2019.
- [18] P. Sahoo, A. K. Singh, S. Saha, V. Jain, S. Mondal, and A. Chadha, “A systematic survey of prompt engineering in large language models: Techniques and applications,” 2025.
- [19] C. Sadasivan, C. Cruz, N. Dolgoy, A. Hyde, S. Campbell, M. McNeely, E. Stroulia, and P. Tandon, “Examining patient engagement in chatbot development approaches for healthy lifestyle and mental wellness interventions: Scoping review,” *J Particip Med*, vol. 15, p. e45772, May 2023.
- [20] M. Sclar, Y. Choi, Y. Tsvetkov, and A. Suhr, “Quantifying language models’ sensitivity to spurious features in prompt design or: How i learned to start worrying about prompt formatting,” 2024.
- [21] C. Bonial and H. T. Madabushi, “A construction grammar corpus of varying schematicity: A dataset for the evaluation of abstractions in language models,” pp. 243–255, 2024.
- [22] B. Nazarov, D. Frolova, Y. Lubarsky, A. Gaissinski, and P. Kisilev, “Rethinking data: Towards better performing domain-specific small language models,” 2025.
- [23] A. M. A. Garg, “Cross-domain content generation with domain-specific small language models,” *ArXiv*, vol. abs/2409.17171, 2024.
- [24] K. Ethayarajh, W. Xu, N. Muennighoff, D. Jurafsky, and D. Kiela, “Kto: Model alignment as prospect theoretic optimization,” 2024.

- [25] W. Zhang and J. Zhang, “Hallucination mitigation for retrieval-augmented large language models: A review,” *Mathematics*, 2025.
- [26] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” 2021.
- [27] K. Shuster, S. Poff, M. Chen, D. Kiela, and J. Weston, “Retrieval augmentation reduces hallucination in conversation,” 2021.
- [28] D. J. Shah, R. Gupta, A. Fayazi, and D. Z. Hakkani-Tür, “Robust zero-shot cross-domain slot filling with example values,” *ArXiv*, vol. abs/1906.06870, 2019.
- [29] R. Pimplikar, K. Kannan, A. Mondal, J. Mondal, S. Saxena, G. Parija, and C. Devulapalli, “Rise: Resolution of identity through similarity establishment on unstructured job descriptions,” pp. 19–36, 2017.
- [30] N. Dixon, M. Goggins, E. Ho, M. Howison, J. Long, E. Northcott, K. Shen, and C. Yeats, “Occupational models from 42 million unstructured job postings,” *Patterns*, vol. 4, no. 7, p. 100757, 2023.
- [31] E. R. Adhikari, “Understanding nuances and commonalities of job descriptions,” vol. 1, pp. 67–76, 2015.
- [32] R. A. Cheramie, “Improving career development in students by developing job analysis skills (forthcoming),” 2014.
- [33] K. Pradeep, “Personal preference in choosing a job,” *International Journal of Innovative Technology and Exploring Engineering*, 2019.
- [34] M. Gan and B. H. Kleiner, “How to write job descriptions effectively,” *Management Research News*, 2005.
- [35] T. Abdelaziz, A. M. Maatuk, and F. Rajab, “An approach to improvement the usability in software products,” *International Journal of Software Engineering Applications*, vol. 7, pp. 11–18, 2016.
- [36] H. Soni, J. Ivanova, H. Wilczewski, A. Bailey, T. Ong, A. Narma, B. E. Bunnell, and B. M. Welch, “Virtual conversational agents versus online forms: Patient experience and preferences for health data collection,” *Frontiers in digital health*, vol. 4, 2022.
- [37] Z. Jiang, F. F. Xu, L. Gao, Z. Sun, Q. Liu, J. Dwivedi-Yu, Y. Yang, J. Callan, and G. Neubig, “Active retrieval augmented generation,” *ArXiv*, vol. abs/2305.06983, 2023.
- [38] S. Sivarajkumar, M. Kelley, A. Samolyk-Mazzanti, S. Visweswaran, and Y. Wang, “An empirical evaluation of prompting strategies for large language models in zero-shot clinical natural language processing: Algorithm development and validation study,” *JMIR Medical Informatics*, vol. 12, 2024.

- [39] Y. Li, R. Ramprasad, and C. Zhang, “A simple but effective approach to improve structured language model output for information extraction,” pp. 5133–5148, 2024.
- [40] G. Mark, V. M. González, and J. Harris, “No task left behind?: examining the nature of fragmented work,” *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, 2005.
- [41] Z. Gou, Z. Shao, Y. Gong, yelong shen, Y. Yang, N. Duan, and W. Chen, “CRITIC: Large language models can self-correct with tool-interactive critiquing,” in *The Twelfth International Conference on Learning Representations*, 2024.
- [42] A. Khan, J. Hughes, D. Valentine, L. Ruis, K. Sachan, A. Radhakrishnan, E. Grefenstette, S. R. Bowman, T. Rocktäschel, and E. Perez, “Debating with more persuasive llms leads to more truthful answers,” 2024.
- [43] L. N. Lior, “Chapter 6 - understanding user reading patterns,” in *Writing for Interaction* (L. N. Lior, ed.), pp. 91–106, Boston: Morgan Kaufmann, 2013.
- [44] Government Digital Service, “Content design: planning, writing and managing content.” <https://www.gov.uk/guidance/content-design>, 2024. Accessed 26 Apr 2025.
- [45] World Wide Web Consortium (W3C), “Web content accessibility guidelines (wcag) 2.2,” Oct. 2023. W3C Recommendation.
- [46] D. Eleyan, A. Othman, and A. Eleyan, “Enhancing software comments readability using flesch reading ease score,” *Inf.*, vol. 11, p. 430, 2020.
- [47] H. Leung, “The impact of information quality of job descriptions on an applicant’s decision to pursue a job,” 2007.
- [48] U. Lagowska, F. Sobral, J. Jacob, A. Hafenbrack, and R. Goldszmidt, “Following community norms or an internal compass? the role of prospective leaders’ social category membership in the differential effects of authentic and ethical leadership on stereotype threat.,” *The Journal of applied psychology*, 2023.
- [49] R. K. Sawyer, “The iterative and improvisational nature of the creative process,” *Journal of Creativity*, vol. 31, p. 100002, 2021.
- [50] J. Schmidhuber, S. Schlogl, and C. Ploder, “Cognitive load and productivity implications in human-chatbot interaction,” in *2021 IEEE 2nd International Conference on Human-Machine Systems (ICHMS)*, p. 1–6, IEEE, Sept. 2021.
- [51] R. Tanaka, A. Ozeki, S. Kato, and A. Lee, “Context and knowledge aware conversational model and system combination for grounded response generation,” *Computer Speech Language*, vol. 62, p. 101070, 2020.

- [52] M. Kleppmann, W. Maughan, and D. Grover, “Upwelling: Combining real-time collaboration with version control for writers.” Ink & Switch Research Lab Technical Report, 2023. Accessed 25 Apr 2025.
- [53] S. Sterman, M. J. Nicholas, and E. Paulos, “Towards creative version control,” *Proc. ACM Hum.-Comput. Interact.*, vol. 6, Nov. 2022.
- [54] J. Chan and C. D. Schunn, “The importance of iteration in creative conceptual combination,” *Cognition*, vol. 145, pp. 104–115, 2015.
- [55] M. Varsha, S. Yashashree, D. K. Ramdas, and S. Alex, “A review of existing approaches to increase the computational speed of the python language,” 2019.
- [56] F. Zehra, M. Javed, D. Khan, and M. Pasha, “Comparative analysis of c++ and python in terms of memory and time,” 2020.
- [57] Y. Zhang, “Comparative study of the execution efficiency of python and c++based on topological sorting,” *Applied and Computational Engineering*, 2024.
- [58] Stack Overflow, “Stack overflow developer survey 2024.” <https://insights.stackoverflow.com/survey/2024>, 2024. Accessed: 17 April 2025.
- [59] J.-S. Lee, T. Filatova, A. Ligmann-Zielinska, B. Hassani-Mahmooei, F. Stonedahl, I. Lorscheid, A. Voinov, J. G. Polhill, Z. Sun, and D. Parker, “The complexities of agent-based modeling output analysis,” *J. Artif. Soc. Soc. Simul.*, vol. 18, 2015.
- [60] Y. Liu, G. Deng, Y. Li, K. Wang, T. Zhang, Y. Liu, H. Wang, Y. Zheng, and Y. Liu, “Prompt injection attack against llm-integrated applications,” *ArXiv*, vol. abs/2306.05499, 2023.
- [61] N. Ligeti-Nagy and Z. G. Yang, “Improve performance of fine-tuning language models with prompting,” *Infocommunications journal*, 2023.
- [62] Pydantic AI Team, “Accessing messages from results.” <https://ai.pydantic.dev/message-history/#accessing-messages-from-results>, 2025. Accessed: 2025-04-24.
- [63] K. T. iunn Ong, T. Kwon, and J. Yeo, “Large language models are self-taught reasoners: Enhancing llm applications via tailored problem-solving demonstrations,” *ArXiv*, vol. abs/2408.12315, 2024.
- [64] A. Drozdov, H. Zhuang, Z. Dai, Z. Qin, R. Rahimi, X. Wang, D. Alon, M. Iyyer, A. McCallum, D. Metzler, and K. Hui, “Parade: Passage ranking using demonstrations with large language models,” *ArXiv*, vol. abs/2310.14408, 2023.

- [65] H. Gao, F. Zhang, W. Jiang, J. Shu, F. Zheng, and H. Wei, “On the noise robustness of in-context learning for text generation,” *ArXiv*, vol. abs/2405.17264, 2024.
- [66] L. Wang, N. Yang, and F. Wei, “Learning to retrieve in-context examples for large language models,” pp. 1752–1767, 2023.
- [67] M. Fang, X. Wan, F. Lu, F. Xing, and K. Zou, “Mathodyssey: Benchmarking mathematical problem-solving skills in large language models using odyssey math data,” *ArXiv*, vol. abs/2406.18321, 2024.
- [68] X. Li, W. Wang, M. Li, J. Guo, Y. Zhang, and F. Feng, “Evaluating mathematical reasoning of large language models: A focus on error identification and correction,” pp. 11316–11360, 2024.
- [69] S. D. Nelson, G. Del Fiol, H. Hanseler, B. I. Crouch, and M. R. Cummins, “Software prototyping: A case report of refining user requirements for a health information exchange dashboard,” *Applied Clinical Informatics*, vol. 7, no. 1, pp. 22–32, 2016.
- [70] N. Upadhyaya, “Low-code/no-code platforms and their impact on traditional software development: A literature review,” 03 2023.
- [71] “React package on npm.” <https://www.npmjs.com/package/react>, 2025. Weekly downloads: 36,131,317.
- [72] Facebook, Inc., “facebook/react: The library for web and native user interfaces.” <https://github.com/facebook/react>, 2025. Stars: 235k.
- [73] Logan, “The most demanded frontend frameworks in 2024.” <https://www.devjobsscanner.com/blog/the-most-demanded-frontend-frameworks/>, 2025. React accounts for 52% of 250k frontend job offers.
- [74] “Routing — next.js documentation.” <https://nextjs.org/docs/pages/building-your-application/routing>, 2025.
- [75] “Deploying next.js to vercel — next.js learn.” <https://nextjs.org/learn/pages-router/deploying-nextjs-app-deploy>, 2025. Deployment finishes in under a minute.
- [76] P. Vaithilingam, E. L. Glassman, J. Inala, and C. Wang, “Dynavis: Dynamically synthesized ui widgets for visualization editing,” *Proceedings of the CHI Conference on Human Factors in Computing Systems*, 2024.
- [77] G. Gonzalez, D. A. S. Moran, S. Houde, J. He, S. I. Ross, M. J. Muller, S. Kunde, and J. D. Weisz, “Collaborative canvas: A tool for exploring llm use in group ideation tasks,” 2024.
- [78] I. Weber, “Large language models are pattern matchers: Editing semi-structured and structured documents with chatgpt,” pp. 3–18, 2024.

- [79] S. Koneru, M. Exel, M. Huck, and J. Niehues, “Contextual refinement of translations: Large language models for sentence and document-level post-editing,” *ArXiv*, vol. abs/2310.14855, 2023.
- [80] P. Laban, J. Vig, M. A. Hearst, C. Xiong, and C.-S. Wu, “Beyond the chat: Executable and verifiable text-editing with LLMs,” in *UIST '24: 37th Annual ACM Symposium on User Interface Software and Technology*, 2024.